

Programação de Sistemas

Programação com strings em C

Marcelo de Gomensoro Malheiros



Recapitulando...



» Criação de vetor com base em elementos

- Um vetor pode ser criado e já inicializado:
 - `int v[10] = {7, 1, 5, 4, 0, 8, 12, -3, 10, 2};`
 - `int len_v = 10;`
 - `for (int i = 0; i < len_v; i++) {`
 - `printf("%i ", v[i]);`
 - `}`
 - `printf("\n");`

v	7	1	5	4	0	8	12	-3	10	2
	0	1	2	3	4	5	6	7	8	9

Recapitulando...

» Limitações dos vetores em C



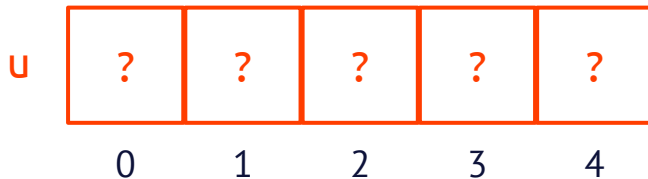
- Operações de listas de Python que não podem em C:
 - teste de pertinência → é preciso percorrer e comparar
 - obter tamanho → é preciso guardar em uma variável
 - adicionar novo elemento → não é possível: vetores de C têm tamanho fixo
 - remover um elemento → não é possível: vetores de C têm tamanho fixo

Recapitulando...

» Criação de vetor não-inicializado

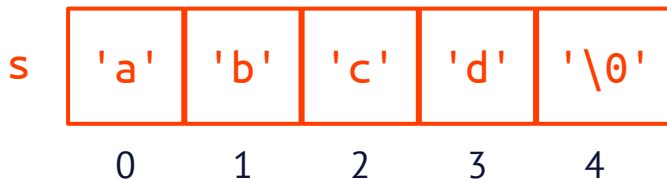


- Basta especificar o seu tamanho (com constante ou variável):
 - `int len_u = 5;`
 - `float u[len_u]; // sem inicialização`
 - `for (int i = 0; i < len_u; i++) {`
 - `u[i] = 0.0;`
 - `}`
 - cada elemento começa com um valor indefinido → é preciso atribuir cada um

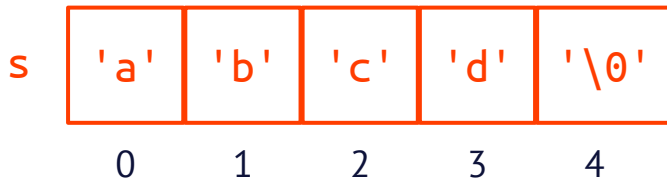


Strings em C

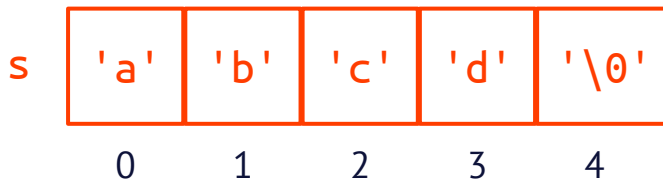
- Uma *string* é simplesmente um vetor de caracteres com um terminador:
 - `char s[] = {'a', 'b', 'c', 'd', '\0'};`
 - os índices sempre começam em 0
 - o terminador `'\0'` sinaliza o final da *string*
 - por simplicidade, podemos omitir o tamanho ao declarar a variável `s[]`



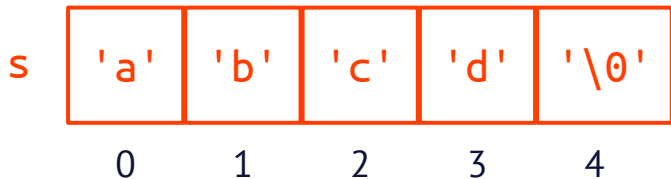
- Por conveniência, uma *string* é especificada com aspas duplas
 - `char s[] = "abcd";`
 - é equivalente a:
 - `char s[] = {'a', 'b', 'c', 'd', '\0'};`
 - note que o terminador **sempre** ocupa espaço no vetor, mas **não conta** em seu tamanho



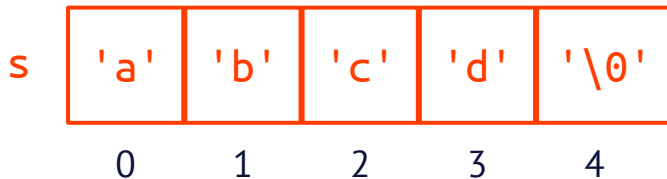
- Os caracteres podem ser acessados pelo seu respectivo índice:
 - `char s[] = "abcd";`
 - `char c = s[0];`
 - `printf("%c\n", c);`
 - $\Rightarrow$ a
 - `printf("%c\n", s[3]);`
 - $\Rightarrow$ d



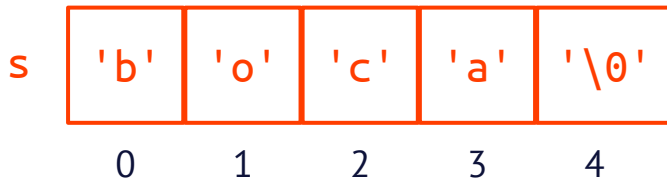
- Uma vez criada, uma *string* pode ser impressa com um laço:
 - `char s[] = "abcd";`
 - `for (int i = 0; s[i] != '\0'; i++) {`
 - `printf("%c", s[i]);`
 - `}`
 - `printf("\n");`
 - $\Rightarrow$ abcd



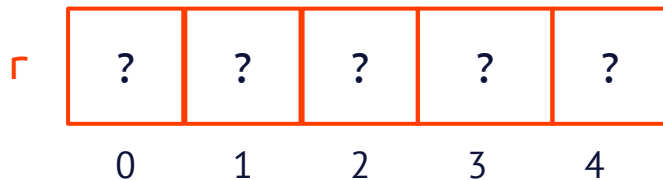
- O mais conveniente é usar o formato `%s` da função `printf()`:
 - `char s[] = "abcd";`
 - `printf("%s\n", s);`
 - $\Rightarrow$ abcd



- Um caracter pode ser alterado com uma atribuição:
 - `char s[] = "bola";`
 - `s[2] = 'c';`
 - `printf("%s\n", s);`
 - $\Rightarrow$ boca

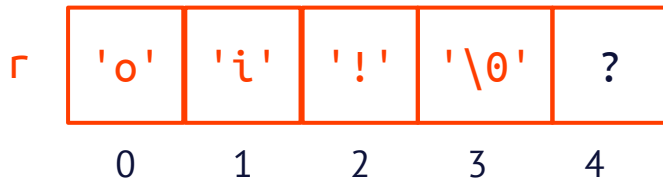


- Uma *string* pode também ser criada apenas com seu tamanho:
 - `char r[5];`
 - cada elemento tem um valor indefinido → é preciso explicitamente definir seu valor com atribuições
 - é **obrigatório** que algum elemento seja o terminador `'\0'`



- O terminador vai indicar onde acaba a *string*:

- `char r[5];`
- `r[0] = 'o';`
- `r[1] = 'i';`
- `r[2] = '!';`
- `r[3] = '\\0';`
- `printf("%s\\n", r);`
- $\Rightarrow$ oi!



- Uma *string* não inicializada pode receber a entrada:
 - `char t[10];`
 - `scanf("%9s", t);` # IMPORTANTE: aceita no máximo 9 caracteres
 - $\Leftarrow$ um dois
 - `printf("%s\n", t);`
 - $\Rightarrow$ um
 - `scanf()` ignora espaços no início do que foi digitado
 - `scanf()` retorna caracteres até o primeiro espaço ou quebra de linha

t	'u'	'm'	'\0'	?	?	?	?	?	?	?
	0	1	2	3	4	5	6	7	8	9

- Uma *string* não inicializada pode receber a entrada:
 - `char t[10];`
 - `fgets(t, 10, stdin);` # IMPORTANTE: aceita no máximo 9 caracteres
 - $\Leftarrow$ um dois
 - `printf("%s\n", t);`
 - $\Rightarrow$ um dois ↵
 - `fgets()` retorna caracteres até a quebra de linha
 - `fgets()` inclui tudo que foi digitado, **inclusive** a quebra de linha ao final

t	'u'	'm'	' '	'd'	'o'	'i'	's'	'\n'	'\0'	?
	0	1	2	3	4	5	6	7	8	9

Funções úteis

- É preciso definir `#include <string.h>` no início do programa
- Tamanho de uma *string*: `strlen("teste")` → 5
- Comparar duas strings: `strcmp("bola", "dado")` → valor negativo
 - a lógica é `("a", "b")` → < 0, `("a", "a")` → 0 e `("b", "a")` → > 0
- Copiar uma string sobre outra: `strcpy(destino, origem)`
 - **atenção:** os tamanhos têm que ser compatíveis!
- Concatenar uma string depois de outra: `strcat(destino, origem)`
 - **atenção:** o tamanho do destino tem que ter espaço sobrando!

Exercícios

- Leia uma *string* e encontre a primeira ocorrência da letra 'a'.
- Leia uma *string* contendo uma frase e então conte o número de palavras, separadas pelo caracter ' '.
- Defina duas *strings* contendo uma palavra cada. Então monte uma nova *string* com as palavras em ordem crescente, separadas pela mensagem " vem antes de ".

Perguntas?